

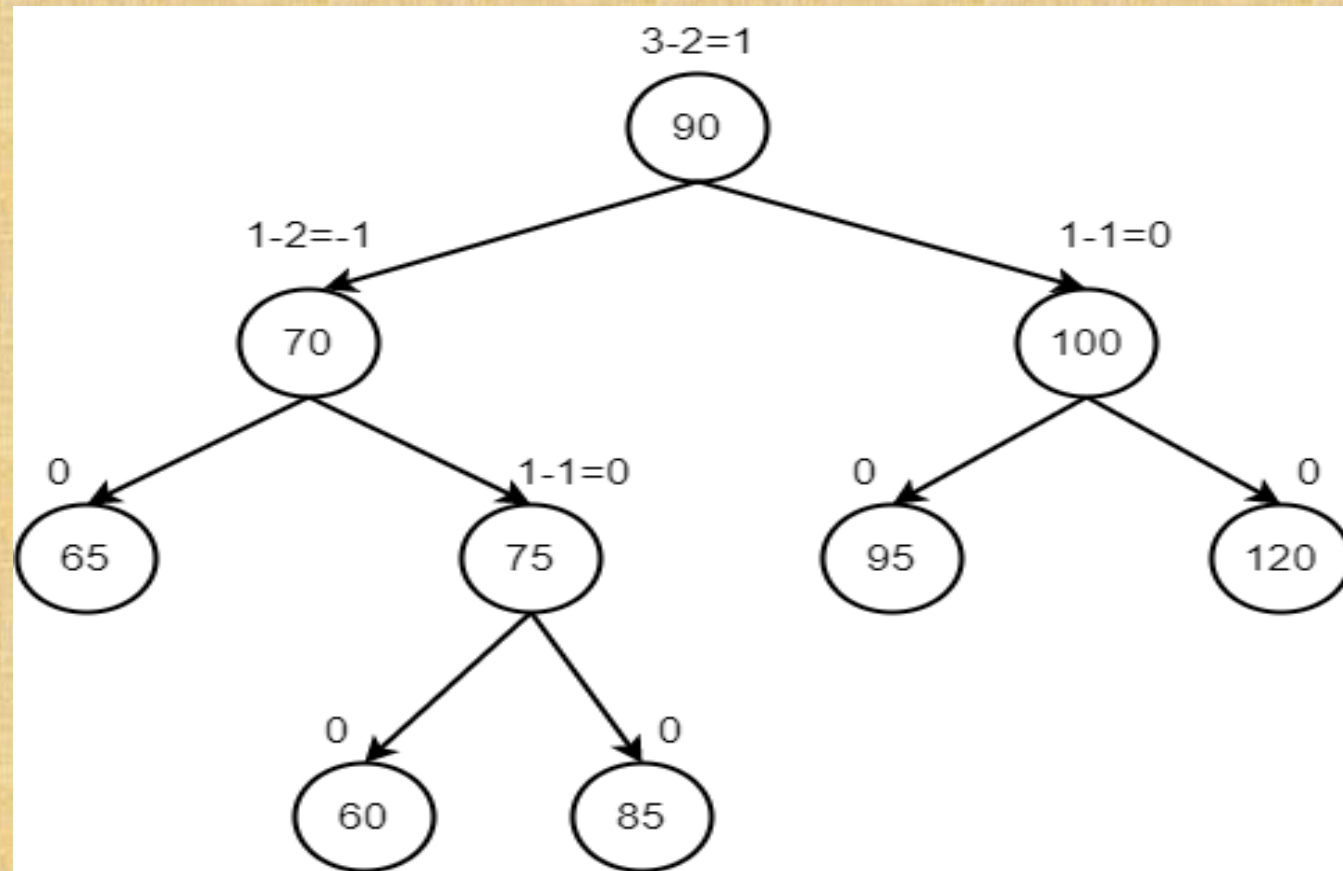
AVL TREES

- **AVL (Adelson-Velsky and Landis) Tree** is a self-balancing binary search tree that can perform certain operations in logarithmic time.
- AVL Tree can be defined as height balanced binary search tree in which each node has a balance factor that is determined by subtracting the height of the node's right subtree from the height of its left subtree.
- Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

Balance Factor (k) = height (left(k)) - height (right(k))

- If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.
- If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.
- If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

Example



Complexity

Algorithm	Average case	Worst case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

Operations on AVL tree

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

Why AVL Tree?

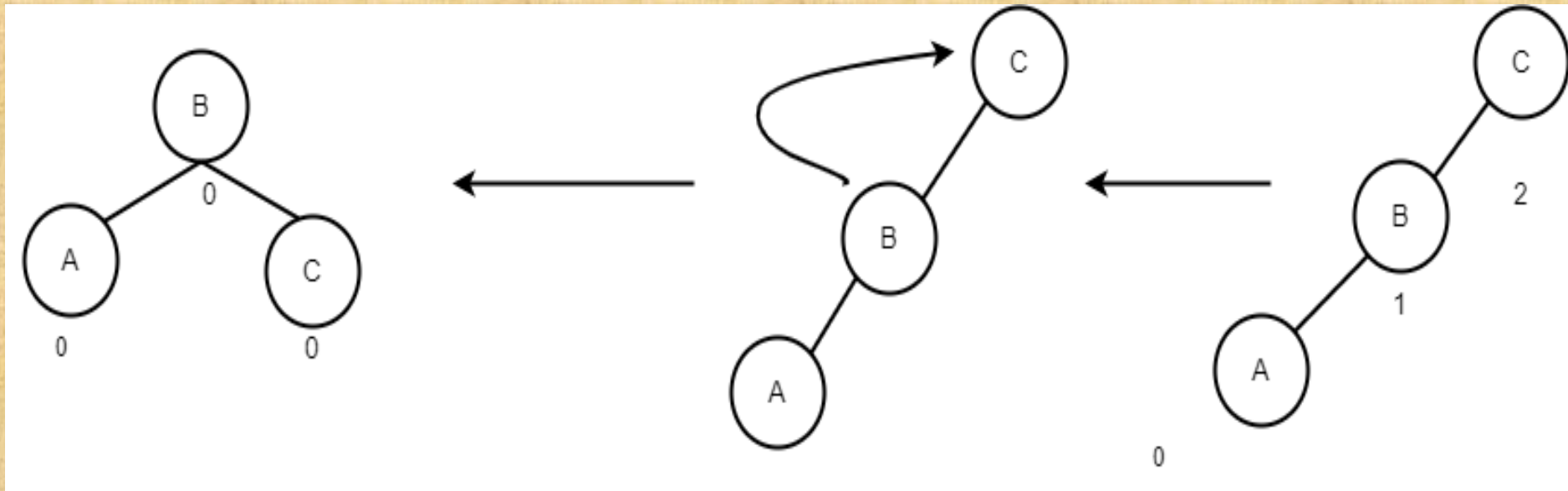
- AVL tree controls the height of the binary search tree by preventing it from becoming skewed.
- In a binary search tree of height h , the total time required for all operations is $O(h)$.
- However, if the BST becomes skewed, it can be extended to $O(n)$ (i.e. worst case).
- The AVL tree puts an upper bound on each operation to be $O(\log n)$, where n is the number of nodes, by limiting this height to $\log n$.

AVL Rotations

There are basically four types of rotations which are as follows:

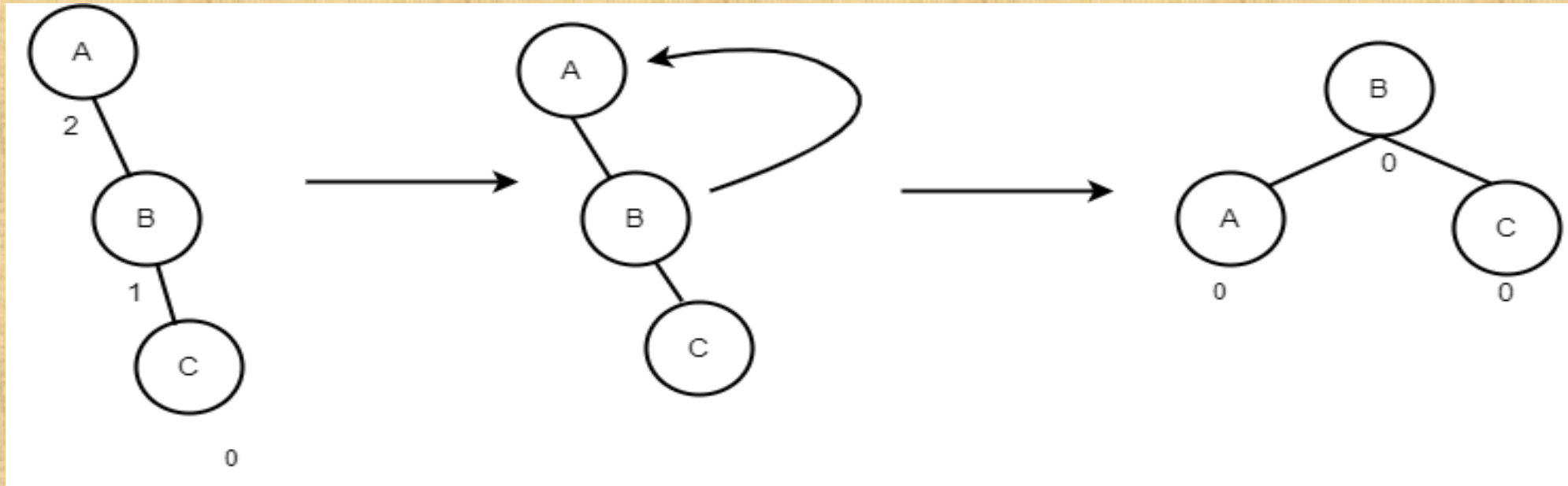
- Right Rotation : Inserted node is in the right subtree of right subtree of A
 - Left Rotation : Inserted node is in the left subtree of left subtree of A
 - Left Right rotation : Inserted node is in the right subtree of left subtree of A
 - Right Left rotation : Inserted node is in the left subtree of right subtree of A
- where node A is the node whose balance factor is other than -1, 0, 1.

Right Rotation



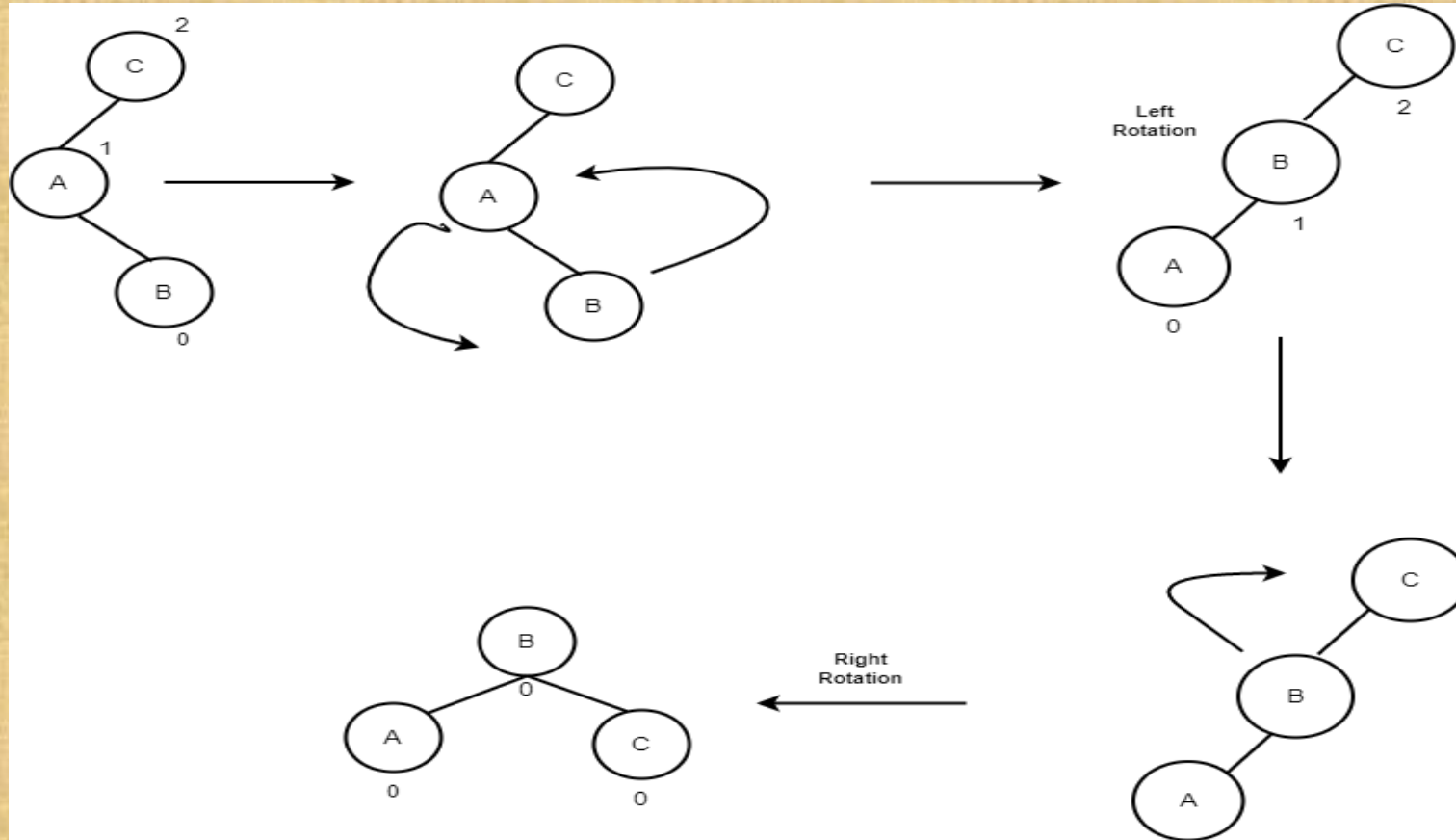
- A single rotation applied when a node is inserted in the left subtree of a left subtree.
- After the addition of node A in the example, node C now has a balancing factor of 2.
- Node B becomes the root when the tree is rotated to the right, creating a balanced tree.

Left Rotation



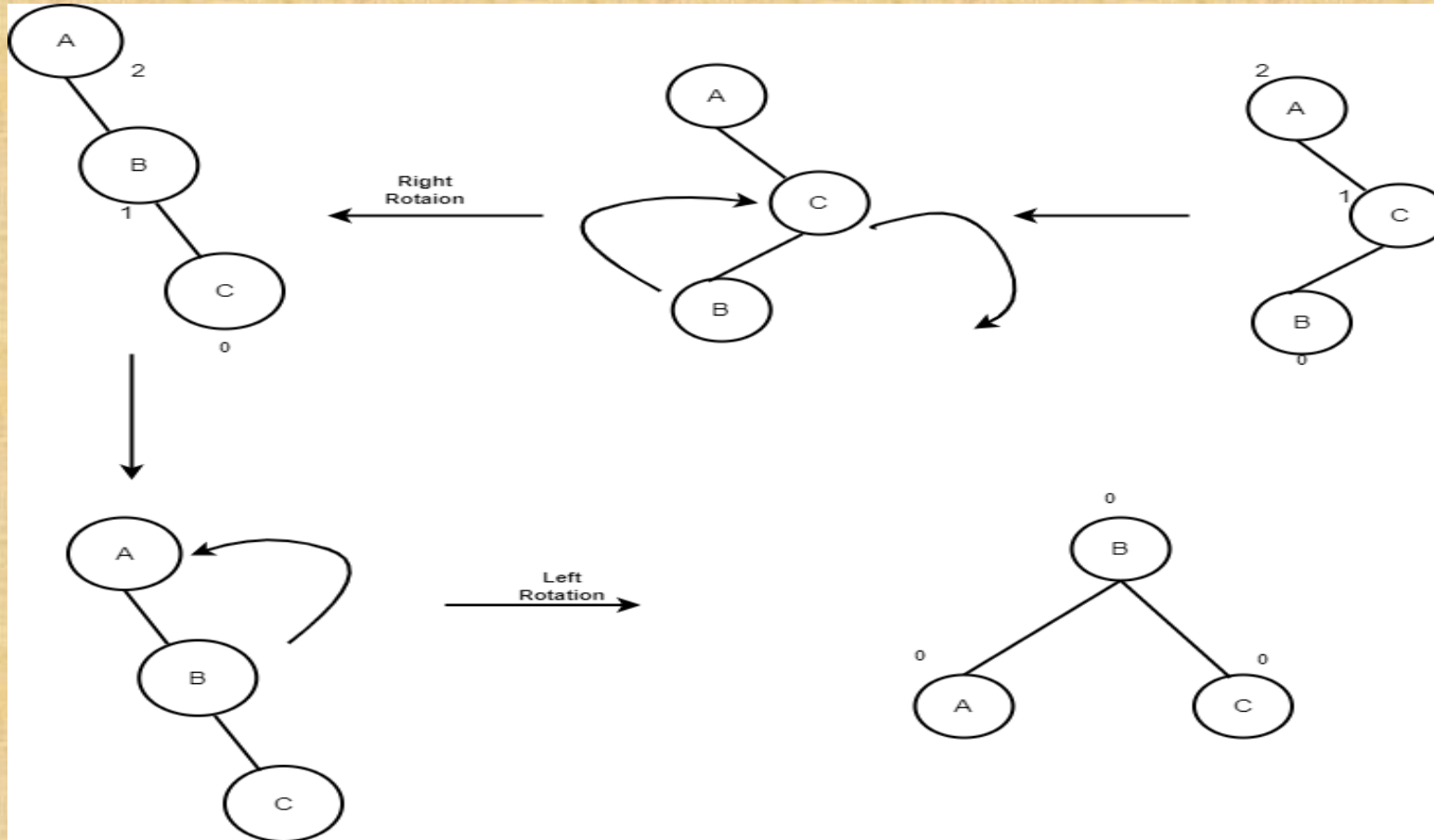
- A single rotation applied when a node is inserted in the right subtree of a right subtree.
- In the given example, node A has a balance factor of 2 after the insertion of node C.
- By rotating the tree left, node B becomes the root resulting in a balanced tree.

LEFT RIGHT ROTATION



- A double rotation in which a left rotation is followed by a right rotation.
- In the given example, node B is causing an imbalance resulting in node C to have a balance factor of 2.
- As node B is inserted in the right subtree of node A, a left rotation needs to be applied.
- A single rotation will not give us the required results.
- Apply the right rotation as shown before to achieve a balanced tree.

Right Left Rotation



- A double rotation in which a right rotation is followed by a left rotation.
- In the given example, node B is causing an imbalance resulting in node A to have a balance factor of 2.
- As node B is inserted in the left subtree of node C, a right rotation needs to be applied.
- By applying the left rotation as shown before, we can achieve a balanced tree.

Advantages of AVL Tree

- AVL Trees were developed to overcome the dependency of operations on the height of the Binary Search Tree and the inability to control the tree's height.
- Similar to Binary Search Trees, it can be used to carry out searching operations on dynamic data sets.
- It always maintains a balanced BST, meaning that its height is minimal, logarithmic, or $O(\log n)$, where n is the total number of nodes in the tree.
- In order to ensure that every node complies with the AVL Balancing requirements, it has a self-balancing property that is accomplished by computing the balance factor of every node after every operation.
- The AVL Tree performs significantly better than BSTs by completing operations in logarithmic time complexity thanks to its self-balancing property, especially when the number of nodes in the tree is very large.
- The searches carried out using AVL trees are faster and more reliable than those of BSTs due to the logarithmic time complexity.